

Parallel Implementation of a CNN Convolutional Layer: A Study using MPI and OpenMP

Raffaele Sinani
mat. 248618
University of Trento
raffaele.sinani@studenti.unitn.it

Damiano Zaccaron
mat. 247952
University of Trento
damiano.zaccaron@studenti.unitn.it

Abstract

Convolutional neural networks are a fundamental model in computer vision, but their computational cost can be significant, especially for large datasets and high-resolution images. To address this challenge, we implement and analyze and compare two parallelized versions of a convolution layer using MPI (Message Passing Interface) and OpenMP to improve performance and scalability.

We then discuss data dependencies, memory access patterns, and the trade-offs between communication overhead and computational gains. Finally, we evaluate our implementations across different matrix sizes and kernel dimensions, analyzing speedup, efficiency, and scalability. Our results demonstrate that the MPI paradigm is not suitable for this task, while OpenMP is able to produce significant performance improvements.

1 Introduction

Convolutional Neural Networks (CNNs) have become a cornerstone in modern deep learning applications, particularly in computer vision tasks such as image classification, object detection, and segmentation. The convolution operation, which extracts spatial hierarchies of features from input images, is computationally intensive, making it a prime target for optimization through parallel computing. As deep learning models grow in complexity and demand larger datasets for training, optimizing these operations becomes crucial to achieving feasible training and inference times.

The computational bottleneck in CNNs primarily arises from the convolutional layers, which involve performing a large number of multiply-accumulate operations across different regions of an image. This motivates the exploration of parallel processing

techniques for CNN layers using CPU-based parallelization frameworks.

In this project, we aim to investigate the performance benefits of parallelizing the convolutional layer of a CNN. We begin with a sequential implementation and incrementally optimize it using two parallel programming models:

- **MPI (Message Passing Interface):** A distributed memory model well-suited for scaling across multiple nodes in an HPC environment. MPI allows processes to communicate efficiently across different machines, making it a natural candidate for large-scale parallelism in convolutional computations.
- **OpenMP (Open Multi-Processing):** A shared memory model that allows efficient parallelization on multi-core processors. OpenMP enables the use of multi-threading to accelerate computations within a single machine by distributing workloads across available cores.

Our primary objectives are to analyze the computational overhead of a sequential CNN convolutional layer and identify its bottlenecks. We also aim to design and implement an MPI-based parallel version that distributes computations across multiple processes to leverage distributed memory architectures. Additionally, we plan to develop an OpenMP implementation that utilizes multi-threading for intra-node parallelism, exploring efficient workload distribution across CPU cores. Finally, we will compare the performance, scalability, and efficiency of the MPI and OpenMP implementations against the sequential baseline and each other, identifying the trade-offs between distributed and shared memory approaches.

Through systematic benchmarking and evaluation, we aim to provide insights into the feasibility and effectiveness of parallelization strategies for deep

learning workloads. Our findings will highlight the strengths and limitations of each approach.

2 Related Works

At the core of a CNN is the convolutional layer, which systematically processes input data using a set of learnable filters to detect relevant features.

This section provides an in-depth examination, explaining their implementation, mathematical formulation, and computational complexity.

2.1 Algorithm

Convolutional layers are the fundamental components of CNNs, designed to extract spatial patterns from input images by applying local receptive fields, thereby reducing the number of learnable parameters compared to fully connected layers. A convolutional layer operates on an input tensor of dimensions $H \times W \times C$, where H and W represent the spatial dimensions and C is the number of input channels. The layer applies F learnable filters (kernels) of size $K \times K \times C$ to produce an output feature map of size $H' \times W' \times F$.

Mathematically, the convolution operation at position (i, j) for a given filter is defined as:

$$Y_{i,j,f} = \sum_{c=0}^{C-1} \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} X_{i+m,j+n,c} \cdot K_{m,n,c,f} + b_f, \quad (1)$$

where $K_{m,n,c,f}$ is the filter weight at position (m, n) for input channel c and output filter f , and b_f is the bias term for filter f . This operation is repeated across all spatial positions of the input, generating multiple feature maps in parallel.

To control the output dimensions, padding and stride are commonly used. Padding extends the borders of the input tensor by adding zero-valued pixels, which prevents excessive shrinkage of feature maps and preserves spatial resolution. For an input of size $H \times W$, a kernel of size $K \times K$, padding P , and stride S , the output dimensions are computed as:

$$H' = \frac{H + 2P - K}{S} + 1, \quad W' = \frac{W + 2P - K}{S} + 1. \quad (2)$$

A stride greater than one reduces the spatial dimensions of the output, acting as an implicit downsampling operation, which helps in reducing computational cost and memory footprint.

Following convolution, a non-linear activation function is applied element-wise to introduce non-linearity into the model. The most commonly

used activation function is the Rectified Linear Unit (ReLU), defined as:

$$f(x) = \max(0, x). \quad (3)$$

ReLU mitigates the vanishing gradient problem and accelerates training convergence. The combination of convolution, bias addition, and activation defines the fundamental operation of CNNs, allowing for hierarchical feature extraction across multiple layers.

2.2 Complexity

The computational complexity of a convolutional layer depends on: the input size $N \times N$, the kernel size $K \times K$, the number of channels C , and the number of filters F . The standard complexity for a single-layer convolution is given by:

$$O(F \cdot C \cdot K^2 \cdot N^2) \quad (4)$$

This complexity highlights the exponential growth in computational cost as the image size or the number of filters increases.

Parallelization can significantly mitigate this cost. In an ideal parallel execution scenario with P processors, the theoretical speedup is given by:

$$S = \frac{O(F \cdot C \cdot K^2 \cdot N^2)}{P} \quad (5)$$

However, factors such as communication overhead and memory access conflicts often limit the actual speedup achieved. Efficient scheduling and load balancing strategies are essential to maximize performance gains.

For the scope of this project, we will focus on using a single channel and a single filter. While this simplifies the implementation, it does not compromise the generalizability or scalability of the results.

In the subsequent sections, we discuss our approach to implementing a parallel CNN convolutional layer using MPI and OpenMP, along with a detailed analysis of performance improvements compared to a sequential implementation.

3 Implementation

3.1 Design choices

For the implementation of our convolutional layer, we made some specific design choices and simplifications. First of all, we chose to utilize randomly generated double-precision matrices, although the most common application of this methodology is to images, which are usually represented by positive integers in the interval $[0, 256]$. This is because

modern implementations of convolutional neural networks include an input normalization layer, which scales values to decimals. Our utilization of doubles, despite the lack of input normalization, should allow better generalization.

Padding is implemented to ensure that the output layer retains the same dimensions as the input, which is a common practice in Convolutional Neural Networks (CNNs). Additionally, padding ensures that edge values of the input matrix contribute equally to the result as the central values. The padded matrix is composed of $Input_rows + 2 * padding$ rows and $Input_columns + 2 * padding$ columns. The original input is placed at the center of this matrix, while the added rows and columns are filled with zeros.

Our choice of activation function fell on the Rectified Linear activation function (ReLU), which is the most common activation function in state-of-the-art neural network implementations. However, the implementation could be easily modified to include different activation functions.

We opted for a stride value of 1, which is, once again, the most common in real world implementations.

Finally, we decided to implement only one input channel, which turns our tensors into $N \times N$ matrices. This corresponds to gray scale images. However, the algorithm is easily generalizable to multidimensional tensors, associated to multiple input color channels.

Kernel and padding sizes have been left as variables that can be modified as needed.

3.2 Sequential implementation

The implementation of the sequential algorithm follows the fundamental kernel convolution process described in Section 2.1. After padding the input matrix, the algorithm iterates over the rows and the columns of the result matrix. For each element it calculates the local value by summing the products of the corresponding elements of the original matrix and the kernel (Pseudocode 3.2).

3.3 Parallel implementation with MPI

3.3.1 The MPI paradigm and MPICH

The Message Passing Interface (MPI) is a standardized and portable message-passing system designed to function on parallel computing architectures. It enables processes to communicate with one another by sending and receiving messages, facilitating the development of parallel applications across diverse computing environments. MPI supports both point-

Algorithm 1 Sequential Algorithm

function MAIN

Initialize Matrix

Initialize PaddedMatrix = PadMatrix(Matrix)

Initialize Kernel

Initialize ResultMatrix

Initialize Sum counter

for Row in ResultMatrix **do**

for Col in ResultMatrix **do**

Set Sum = Bias

for KRow in Kernel **do**

for KCol in Kernel **do**

$Sum + = PaddedMatrix[Row + KRow][Col + KCol] * kernel[KRow][KCol];$

end for

end for

Set ResultMatrix[Row][Col]

$= \text{ReLU}(\text{Sum})$

end for

end for

end function

to-point and collective communication, aiming for high performance, scalability, and portability.

In particular, our work used MPICH, a high-performance, widely portable implementation of the MPI standard.

3.3.2 MPI Functions utilized

The MPICH implementation of the MPI standards comes with some useful communication functions. We are going to discuss the ones that we utilized for our parallel program.

- **MPI.Bcast** : The MPI_Bcast is the basic broadcasting function of MPICH. It allows the transfer of an input buffer from a master node to every other node in the communicator.
- **MPI.Scatterv**: The MPI_Scatterv function scatters a buffer in parts to all processes in a communicator, according to the sendcounts argument of the function. This was preferred to the regular MPI_Scatter function because it allows our application to handle a number of elements that is not divisible by the number of processes.
- **MPI.Reduce**: This function reduces values on all processes to a single value. In our case we utilized the MPI_SUM operation to sum all result vectors back to the master node.
- **MPI.Wtime**: This function, alongside MPI_Barrier allows us to take precise measurements of our application performance.

It is worth noting that all MPICH communication functions utilize a tree-based implementation, this allows to reduce the complexity to $\log_2(n)$, where n is the number of processes.

3.3.3 Parallel implementation

To apply parallel computation to our problem, we had to apply some modification to the original algorithm. First of all, all matrices had to be substituted by their row-wise flattened vector, which is obtained by placing their rows one after the other. This is due to the way matrices are implemented and stored when using the C programming language. As matrices are pointers to pointers they cannot be used directly as buffers for the MPI communication functions.

Furthermore, utilizing the same algorithm we used for the sequential implementation would require sending the whole matrix to every node, bar a massive increase in code and communication complexity. That is because many rows of the matrix would need to be sent to multiple different nodes, as calculating the value of an element of the result matrix requires having the previous and successive rows and columns of the original matrix.

Therefore, we opted for a different approach for the MPI version of the problem (Pseudocode 2). We decided to follow a well documented approach to similar problems that used row partitions. The rows of the original matrix would be divided evenly across the nodes (or as evenly as possible in case the number of rows is not divisible by the number of processes) using MPI.Scatterv. In practice, this would amount to sending a portion of the row-wise flattened vector. We would then apply sequence convolution to the portion of the vector on every node and the row-wise flattened kernel, summing each product to the correct place in the result vector.

Calculating the original row and column index is used as an intermediate step to calculating the right spot in the row-wise flattened local result vector. It is also used inside an if condition to ensure that the sums are never added out of bounds, as some of the sums are not supposed to be used for the end result, i.e. the first element of the matrix and the last element of the kernel.

The result vector of each process is then summed using MPI.Reduce and finally the bias is summed and the activation function is applied.

This approach, still presents a significant issue:

Algorithm 2 Parallel Algorithm using MPI

```

function MAIN
  if ProcessRank = 0 then
    Initialize Matrix
    Initialize PaddedMatrix = PadMa-
    trix(Matrix)
    Initialize Kernel
    Initialize FlatMat = flattenMa-
    trix(PaddedMatrix)
    Initialize FlatKer = flattenMatrix(Kernel)
    Initialize ResultVector
  end if
  Initialize SendCounts, Displacements
  Initialize LocalMatrix, LocalRes
  MPI_Bcast (FlatKer)
  MPI_Scatterv (FlatMat, SendCounts,
    Displacements, LocalMat)
  for El in LocalMat do
    for KEI in FlatKer do
      Find ResultIndex
      if ResultIndex  $\in [0, ResultSize]$  then
        LocalRes[ResIndex] +=
          LocalMat[El]  $\times$  FlatKer[KEI]

      end if
    end for
  end for
  MPI_Reduce(LocalRes, Result)
  for Element in Result do
    Set Element = ReLU(Element + bias)
  end for
end function

```

the result vector, which is comparable in size to the original matrix still needs to be reduced across the cluster. However, this approach yielded similar results to more complex methods that we experimented with. One such methods consisted in sending different portions of the original matrix to different nodes and utilizing the original algorithm. This complicated the broadcasting communication of the original matrix, while still retaining larger-than-ideal messages. Another approach we experimented included sending all multiplication results to the master node, along with their coordinates in the result matrix. While this reduced individual message sizes, it significantly increased the total number of messages, creating additional communication overhead.

Ultimately, we concluded that the core issue lies in the task itself, as it is impossible to solve this problem on multiple processes without a shared memory space, without substantial inter-process communication.

3.4 Parallel Implementation with OpenMP

3.4.1 The OpenMP paradigm

Open Multi-Processing (OpenMP) is an application programming interface (API) that enables shared-memory parallelism. Designed to simplify parallel computing, OpenMP allows developers to write multithreaded applications with minimal modifications to existing sequential code. It provides a standardized approach to creating parallel regions, distributing workloads, and synchronizing threads efficiently. OpenMP is an implementation of multithreading, a method of parallelizing whereby a primary thread (a series of instructions executed consecutively) forks a specified number of sub-threads and the system divides a task among them.

A fundamental aspect of OpenMP is the use of compiler directives to define parallel regions, such as `#pragma omp parallel`, which allows code execution across multiple threads. Work-sharing constructs, including parallel loops, divide computational tasks among threads to improve load balancing.

3.4.2 Data Dependencies

Data dependency analysis in parallel code context is used to identify and manage dependencies between different parts of a program that may affect execution order. A dependency occurs when one statement reads from or writes to a memory location that another statement also accesses, and at least one of these operations is a write. This analysis is crucial for parallel programming because it helps determine which parts of the code can be executed concurrently without causing race conditions or inconsistencies.

In our code, potential dependencies and challenges to parallelization arise from the convolution operation. Therefore, we focus on the relevant section of the code and its dynamic extent:

```

1  for (i = 0; i < result_rows; i++) {
2      for (j = 0; j < result_cols; j++) {
3          double sum = bias;
4          for (k = 0; k < ker_size; k++) {
5              for (l = 0; l < ker_size; l++) {
6                  int row = i + k;
7                  int col = j + l;
8                  sum += padded_matrix[row][col] * kernel[k][l];
9              }
10         }
11         result[i][j] = relu(sum);
12     }

```

```

13 double relu(double x) {
14     return (x > 0.0) ? x : 0.0;
15 }
16 }
17 }

```

Listing 1: Convolution Operation in the CNN Layer

The code contains four for loops, each a potential candidate for parallelization. In the Table below (see Table 1), we analyze data dependencies between parts of the code. The loops are paired into two groups: the outer loops (i and j) and the inner loops (k and l). Additionally, Table 2 shows data dependencies that would exist in the algorithm but have already been resolved in our code. This is due to our choice of initializing the three variables inside their respective loops, limiting their scope and ensuring that during different loop iterations, data is not written to the same memory location. By handling dependencies this way, no further modifications are needed when implementing OpenMP parallelization.

It can be noticed that the ReLU activation function is very simple and does not cause any data dependency. However, it has still been added to provide a complete outlook to the dynamic extent of the code.

Memory Locations	Earlier Statement			Later Statement			Loop Carried?	Kind of Statement
	Line	Iteration	Access	Line	Iteration	Access		
sum	3	(i, j)	write	8	(i, j)	read	No	Flow
sum	3	(i, j)	write	8	(i, j)	write	No	Output
sum	8	(k, l)	write	8	(k+1, l+1)	read	Yes (Inner Loop)	Flow
sum	8	(i, j)	write	11	(i, j)	read	No	Flow
row	6	(j, k)	write	8	(j, k)	read	No	Flow
col	7	(j, k)	write	8	(j, k)	read	No	Flow

Table 1: Remaining Data Dependencies

Memory Locations	Earlier Statement			Later Statement			Loop Carried?	Kind of Statement
	Line	Iteration	Access	Line	Iteration	Access		
sum	8	(i, j)	write	3	(i+1, j+1)	write	Yes (Outer Loop)	Output
row	6	(j, k)	write	6	(j+1, k+1)	write	Yes (Inner Loop)	Output
col	7	(j, k)	write	7	(j+1, k+1)	write	Yes (Inner Loop)	Output

Table 2: Solved Data Dependencies

Analyzing the dependencies, a choice has been made to only parallelize the two outermost cycles. This is because they iterate over independent output pixels—each iteration computes a distinct element of the result matrix without overlapping data. In contrast, parallelizing the innermost cycles, which handle the accumulation of products for each output pixel, would introduce data dependencies on the accumulation variable (i.e., the variable `sum`).

In fact, multiple threads attempting to update `sum` concurrently would lead to race conditions, requiring the use of reduction clauses or critical sections. Such synchronization mechanisms would incur significant overhead and could negate the benefits of parallelization.

To resolve the data dependency, the variables `k` and `l` are set as private. The variables `sum`, `row` and `col` are also set as private, as they are initialized within the loop.

The code is therefore modified accordingly:

```

1 #pragma omp parallel for
  collapse(2) private(k, l)
  schedule(static)
2 for (x = 0; x < result_rows; x++) {
3   for (y = 0; y < result_cols;
4     y++) {
5     double sum = bias;
6     for (k = 0; k < KER_SIZE;
7       k++) {
8       for (l = 0; l < KER_SIZE;
9         l++) {
10        int row = x + k;
11        int col = y + l;
12        sum +=
13          padded_input[row][col]
14          * kernel[k][l];
15      }
16    }
17    result[x][y] = relu(sum);
18  }
19 }

double relu(double x) {
  return (x > 0.0) ? x : 0.0;
}

```

Listing 2: Parallelised Convolution Operation in the CNN Layer

3.4.3 Implementation

Our implementation of the convolutional layer using OpenMP focuses on parallelizing the computationally intensive convolution operation while ensuring correct memory management and efficient execution.

The parallelization is focused on the most resource-intensive part of the sequential code. In fact, OpenMP directives are used within the nested loops that iterate over the *output* feature map.

Specifically, the outermost loops are parallelized using `#pragma omp parallel for collapse(2) private(k,l) schedule(static)`. This allows multiple threads to compute different regions of the output matrix concurrently, reducing computation time significantly.

The `collapse(2)` clause enables the compiler to

merge loop iterations, further improving workload distribution across threads.

The `private(k,l)` is particularly necessary because these variables are declared earlier in the code, and the GCC compiler does not support C99-style loop variable declarations by default.

The variables `sum`, `row`, `col` are declared inside the loop, so they are automatically set as private. The `schedule(static)` ensures an even distribution of workload among threads, avoiding thread imbalance and excessive synchronization overhead.

Finally, the implementation incorporates ReLU activation, ensuring that only positive values contribute to the final output.

The function `omp_get_wtime()` is used to provide precise timing measurements to evaluate the performance of the parallel implementation. Multiple iterations are executed to determine the minimum execution time for reliable benchmarking.

Algorithm 3 Parallel Convolution using OpenMP

```

1: function MAIN
2:   Initialize Matrix
3:   Initialize PaddedMatrix  $\leftarrow$  PadMa-
      trix(Matrix)
4:   Initialize Kernel
5:   for each row  $r$  in Matrix do
6:     for each column  $c$  in Matrix do
7:       PaddedMatrix[ $r + \text{Padding}$ ][ $c +$ 
8:         Padding]  $\leftarrow$  Matrix[ $r$ ][ $c$ ]
9:     end for
10:  end for
11:  #pragma omp parallel for collapse(2)
12:  private(k, l) schedule(static)
13:  for each row  $r$  in ResultMatrix do
14:    for each column  $c$  in ResultMatrix do
15:      sum  $\leftarrow$  bias
16:      for each  $k_r$  in Kernel do
17:        for each  $k_c$  in Kernel do
18:          sum  $\leftarrow$  sum +
19:            PaddedMatrix[ $r + k_r$ ][ $c + k_c$ ]  $\times$  Kernel[ $k_r$ ][ $k_c$ ]
20:        end for
21:      end for
22:      ResultMatrix[ $r$ ][ $c$ ]  $\leftarrow$  ReLU(sum)
23:    end for
24:  end for
25: end function

```

4 Benchmarks and Performance

4.1 HPC Cluster Unitrento

The benchmarks for this project have been executed on the Unitn HPC Cluster. The cluster relies on

the cluster management software Altaris PBS Professional and the operating system of the nodes is Linux CenOS7. the HPC cluster is composed of:

- nodes: 126
- CPU cores: 6092
- CUDA cores: 37376
- RAM: 53 TB

All nodes are interconnected with 10Gb/s network, some have also Infiniband 40Gb/s connectivity and others have Omnipath 100 Gb/s connectivity. The cluster is also connected to a high-performance shared storage (15 TB) backed up daily in a secondary data center.

4.2 PBS directives

To run our applications on the HPC cluster we utilized the following PBS instructions.

```

1 #!/bin/bash
2
3 #PBS -l select=1:ncpus=N:mem=256gb
4
5 #PBS -l walltime=01:05:30
6
7 #PBS -q short_cpuQ
8
9 export OMP_NUM_THREADS=N
10 /home/name.surname/ParallelCNNLayer/
11 src/openmp $KER_SIZE $MAX_SIZE

```

Listing 3: OpenMP PBS

This is the code used for the OpenMP application. The first line indicates the number of nodes we require and the amount of memory. The second line sets the maximum execution time and the third line the queue we are utilizing. Finally, the **export** directive is used to bring an argument inside the openMP parallel paradigm. Our code accepts two arguments: one is used to decide the maximum matrix size, which is calculated via $pow(2, MAX_SIZE)$, while the other specifies the number of rows and columns of the kernel.

The MPI PBS submission script presents just a few differences:

```

1 #PBS -l select=N:ncpus=1:mem=64gb
2 #-l place=scatter:excl
3
4 module load mpich-3.2
5 mpirun.actual -n N
6 /home/name.surname/
7 ParallelCNNLayer/src/mpi_z
8 $KER_SIZE $MAX_SIZE

```

Listing 4: MPI PBS

The first line has been modified to highlight our choice of placement strategy. We chose to utilize a scatter-exclusive strategy, which allows us to reserve a whole node for each instance of our parallel application, allowing for more consistent benchmarking. This was not possible for number of nodes superior to 32, where we were forced to switch to a scatter policy. The **N** should be substituted with the number of cores we want to dedicate to the application (Listing 3).

The *module load* command ensures that the MPICH package is loaded before running the application. The *mpirun* command is also required to execute an MPI application. Finally, the number of nodes **N** should also be indicated in the last line to allow the application to know the number of nodes in use (Listing 4).

4.3 Performance metrics

In high-performance computing, *speedup* and *efficiency* are fundamental metrics used to evaluate the performance of parallel computing systems. These measures help assess how effectively a system utilizes its computational resources and scales with increasing processing power.

Speedup quantifies the performance gain achieved by parallel execution compared to sequential execution. It is defined as:

$$S(p) = \frac{T_1}{T_p} \quad (6)$$

where:

- T_1 is the execution time of the sequential algorithm,
- T_p is the execution time using p processors.

Efficiency measures how well computational resources are utilized in a parallel system. It is defined as:

$$E(p) = \frac{S(p)}{p} = \frac{T_1}{p \cdot T_p} \quad (7)$$

Efficiency values range between 0 and 1 (or 0% to 100%), where a higher efficiency indicates better resource utilization. Ideally, $E(p) \approx 1$, meaning the workload is evenly distributed across all processors. However, due to communication overhead, synchronization, and load imbalance, efficiency tends to decrease as p increases.

4.4 MPI Evaluation

As expected the MPI application does not lead to acceptable results as shown in Tables 3 and 4. For

smaller kernels, efficiency decreases with the number of cores due to the significant increase in overhead. Better results are reached with larger kernels, as the size of the kernel increases exponentially the number of computations executed by every node, making the communication problems less noticeable. However, this approach would lead to a distortion of the original problem, as convolutional neural networks are usually composed of much smaller kernels. This suggests that communication overhead vastly exceeds the computational workload per node.

Our benchmarks show an efficiency that stabilizes in the 0.25 to 0.3 range, with a few exceptions (Figure 4.4). We can observe a decrease in performance at high core counts, where the small kernel size, paired with the large messages, can lead to inefficiency. However, it must be noted that the results obtained for 64 and 128 cores can be unreliable as the different placement strategy required at such scales can lead to major fluctuation in benchmark performance. This could be part of the reason behind the much lower performance at 64 cores.

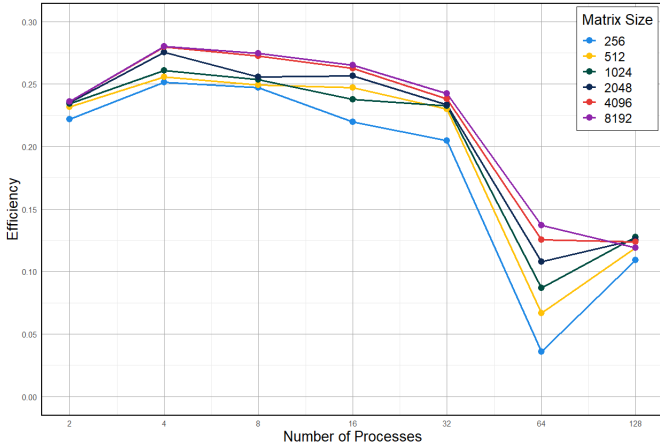


Figure 1: Efficiency evolution for different matrix dimensions with kernel size 25 using MPI

The results obtained are comparable to the only literature work facing a similar problem ([3]). They reached an efficiency of 0.375 on 8 nodes, applied to the problem of sequence convolution, showing a much better performance for long sequences.

Furthermore, there is also an issue arising from the maximum size of the buffer that is allowed for MPI.Send communication, which is 2GB. This limit is easily surpassed by $16384 (2^{14}) \times 16384$ matrices. Tackling this issue would require further complications in the communication structure, as the buffers would need to be split, bar using a completely different algorithm. For smaller kernel sizes, an MPI approach is highly discouraged. On the one hand,

it does not make much sense to apply it to smaller matrices, as the execution time is already low. On the other hand, when utilizing larger matrices the amount of overhead would outweigh the small number of computations required by a small kernel.

In conclusion, unless we face a matrix convolution problem, with a filter of comparable size to the original matrix, a different approach should be utilized, such as OpenMP, which exploits the advantages of a shared memory space to solve the issue of communication. This conclusion is corroborated by the little amount of literature existing that tries to solve similar problem with an MPI approach.

Matrix Size	Number of Cores						
	2	4	8	16	32	64	128
64	0.1925	0.1981	0.1908	0.1572	0.1374	0.0062	0.0651
128	0.2032	0.2237	0.2187	0.1989	0.1723	0.0166	0.0843
256	0.2218	0.2516	0.2470	0.2198	0.2045	0.0356	0.1091
512	0.2316	0.2557	0.2494	0.2471	0.2299	0.0668	0.1191
1024	0.2341	0.2610	0.2535	0.2376	0.2327	0.0868	0.1274
2048	0.2352	0.2756	0.2559	0.2566	0.2334	0.1078	0.1255
4096	0.2358	0.2797	0.2722	0.2626	0.2381	0.1254	0.1237
8192	0.2360	0.2801	0.2747	0.2651	0.2425	0.1370	0.1192

Table 3: Efficiency for Kernel size 25.

Matrix Size	Number of Cores						
	2	4	8	16	32	64	128
64	0.2233	0.2315	0.2216	0.2001	0.1735	0.0237	0.1308
128	0.2097	0.2205	0.2232	0.2171	0.2053	0.0493	0.1181
256	0.2172	0.2387	0.2368	0.2386	0.2223	0.0719	0.1478
512	0.2311	0.2586	0.2609	0.2617	0.2513	0.1008	0.1747
1024	0.2334	0.2710	0.2644	0.2665	0.2612	0.1115	0.1824
2048	0.2358	0.2778	0.2712	0.2756	0.2659	0.1553	0.1916
4096	0.2373	0.2822	0.2806	0.2786	0.2708	0.1641	0.1951
8192	0.2378	0.2839	0.2824	0.2802	0.2739	0.1626	0.1990

Table 4: Efficiency for Kernel size 51.

4.5 OpenMP Performance

4.5.1 Choice of kernel size

In the same manner as our MPI implementation, we tested our OpenMP implementation with multiple kernel sizes. However, unlike MPI, we observed no significant improvement in application performance for any kernel size (Figure 4.5.1). Therefore, our primary analysis focuses on a kernel size of 25. This choice represents a reasonably small yet practical kernel size for our original problem while maintaining a longer execution time, which helps illustrate performance trends more clearly.

For completeness, we also present results for smaller kernel sizes, though they exhibit greater variability, particularly when combined with smaller matrix sizes. For this reason, we excluded very small matrices from our analysis, as their low execution

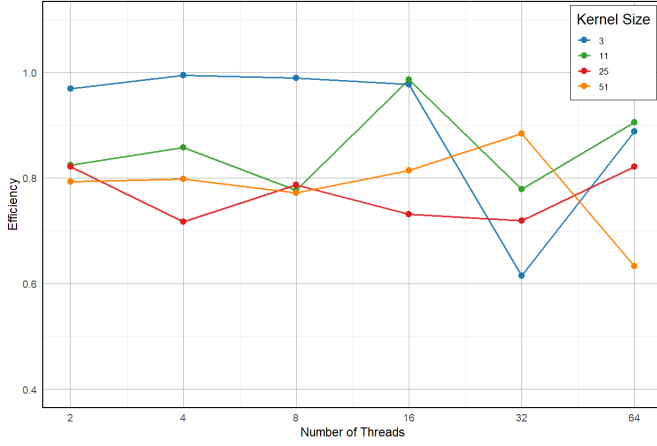


Figure 2: Comparison of efficiency for different kernel sizes using OpenMP

times in both sequential and parallel executions introduce significant performance fluctuations.

It is worth noting that the algorithm performs similarly even with much smaller kernel sizes, such as 3, which is commonly used in practical applications.

4.5.2 Performance

As shown in Figure 4.5.2 the speedup increases almost linearly with the number of threads. This behavior should be expected, as distributing the load across processes should lead to an increase in performance. Furthermore, our choice of matrix size allows for a perfect division of the load among the threads. A different choice of matrix size, i.e. with an odd number of rows, might lead to slight decreases in performance, as the workload cannot be split evenly.

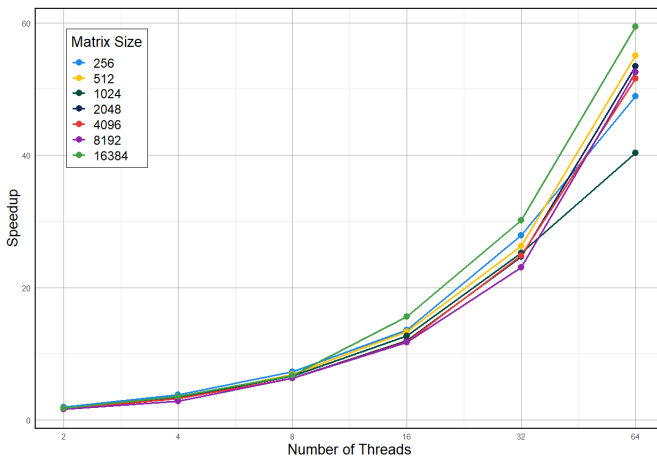


Figure 3: Speedup evolution for different matrix dimensions with kernel size 25 using OpenMP

Overall, there seems to be no noticeable variation in the trend for larger matrix sizes and larger numbers of cores. This can be explained by our

choice of parallelization algorithm, that contains no race condition, which allows our parallel application to run smoothly.

We can now proceed with an analysis of the efficiency, as shown in Figure 4.5.2. The plot shows promising results, as our application seems to be running efficiently. However, we can identify a few outliers.

First of all, the matrix of size 256 seems to be showing slightly better performances for low process numbers. This can probably be attributed to the already short execution time, that might lead to larger fluctuations. The second concern regards increased performance for the largest matrix size at higher process numbers. This could be partially explained by the lower impact of communication. The other factor could be the different performance of allocated nodes on the cluster at different times.

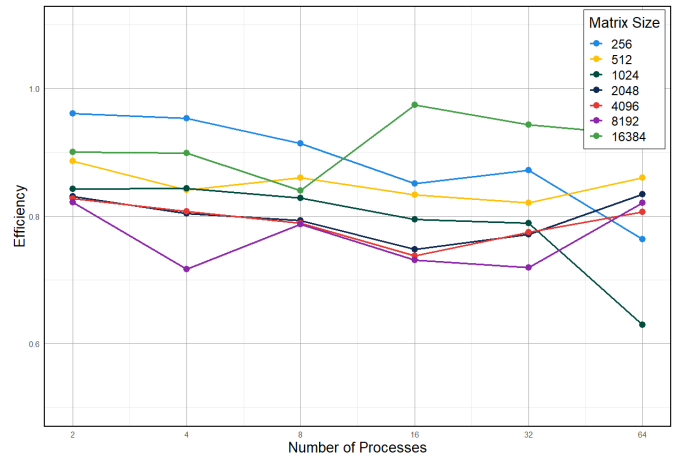


Figure 4: Efficiency evolution for different matrix dimensions with kernel size 25 using OpenMP

Overall, the performance is satisfactory, as the efficiency stays around 0.8 (Table 5), with a few fluctuations below and above. Generally, we can identify a slight downwards trend until 16 threads, which is to be expected due to the increase in communications, however small, that follows the increased number of threads. This result suggests our program to be fairly scalable when utilizing the OpenMP framework.

Finally, as mentioned earlier, this program demonstrates similar performance with smaller kernel sizes. Figure 4.5.2 presents the speedup achieved for an 11×11 kernel. As expected, efficiency fluctuates more for smaller matrices (256 and 512) due to their significantly shorter execution times. However, for larger matrices the performance remains comparable to that of the larger kernels. This shows that the OpenMP implementation could be well-suited to real-world scenarios where parallelization would be needed, such as processing large images with high

Matrix Size	Number of Cores					
	2	4	8	16	32	64
256	0.961	0.953	0.914	0.851	0.871	0.764
512	0.886	0.841	0.860	0.833	0.820	0.860
1024	0.842	0.843	0.828	0.795	0.789	0.630
2048	0.831	0.804	0.793	0.748	0.771	0.834
4096	0.827	0.807	0.789	0.737	0.774	0.806
8192	0.821	0.717	0.787	0.731	0.719	0.821
16384	0.900	0.899	0.840	0.975	0.943	0.793

Table 5: Efficiency for Kernel size 25 with OpenMP.

sequential execution times.

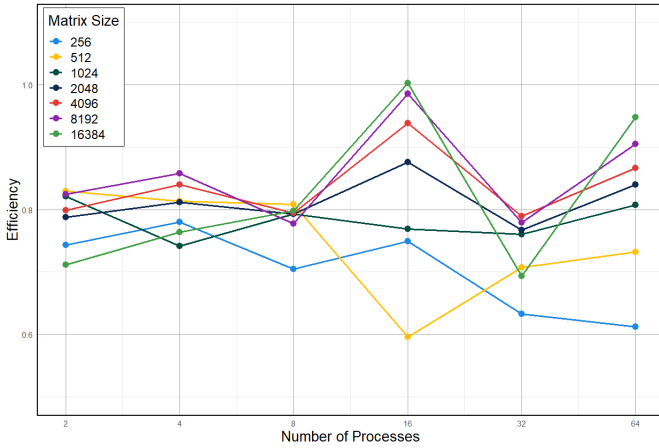


Figure 5: Efficiency evolution for different matrix dimensions with kernel size 11 using OpenMP

4.5.3 Comparisons

This analysis confirms our expectations regarding our two methodologies. OpenMP performs much better in every situation compared to MPI. The stark difference can be observed in Figure 4.5.3. This plot shows the average efficiency across different matrix sizes at kernel size 25, and it is clear to see that OpenMP performs much better all across the board. This advantage is further amplified with small kernels, which is much closer to a real case scenario, as the lower number of computations per thread emphasizes the excessive communication of MPI.

5 Conclusions

This work aims to analyze the matrix convolution problem in Convolutional Neural Networks, and explores possible parallelization strategies using the MPI and OpenMP paradigms.

Our results clearly show that the MPI approach is not suitable for this problem, due to the massive overhead communication problems. However, the OpenMP implementation demonstrated significantly

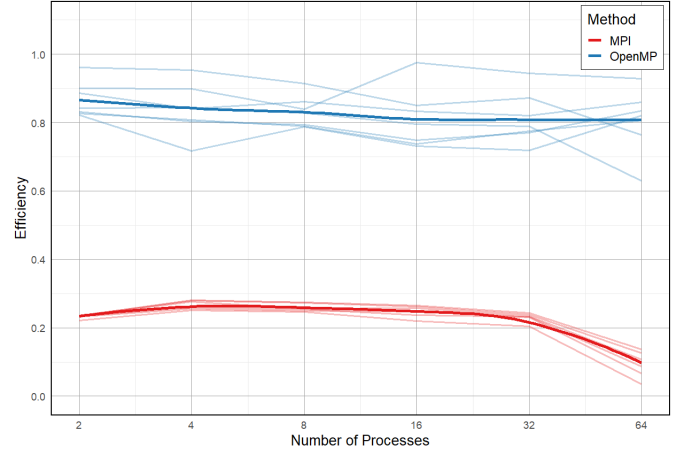


Figure 6: Comparison of efficiency for kernel size 25 between MPI and OpenMP

better performance, leveraging shared memory to minimize communication overhead and consistently outperforming MPI. This suggests that a hybrid approach would offer little to no advantage over a purely OpenMP-based solution.

In conclusion, the results obtained with OpenMP provide significant improvements to the sequential algorithm, showcasing its effectiveness in parallel processing. Moreover, we can affirm that results suggest good scalability even for high matrix dimensions. However, due to technological limitations of CPU-based architectures, they still cannot compare to state-of-the-art implementations, that achieve much better results using GPU programming [4]. These findings suggest that while OpenMP provides a viable path for parallelization, further advancements may require exploring GPU-based solutions to fully unlock the potential of modern high-performance computing.

References

- [1] J. Huang, T. Wang, X. Zhu, *et al.*, “A parallel optimization of the fast algorithm of convolution neural network on cpu,” in *2018 10th International Conference on Measuring Technology and Mechatronics Automation (ICMTMA)*, 2018, pp. 5–9. DOI: 10.1109/ICMTMA.2018.00009.
- [2] S. Lee, D. Jha, A. Agrawal, A. Choudhary, and W.-k. Liao, “Parallel deep convolutional neural network training by exploiting the overlapping of computation and communication,” in *2017 IEEE 24th International Conference on High Performance Computing (HiPC)*, 2017, pp. 183–192. DOI: 10.1109/HiPC.2017.00030.

- [3] J. Lu, K. Zhang, M. Chen, and K. Ma, "Implementation of parallel convolution based on mpi," in *Proceedings of 2013 3rd International Conference on Computer Science and Network Technology*, 2013, pp. 28–31. DOI: 10.1109/ICCSNT.2013.6967057.
- [4] Y.-b. Huang, K. Li, G. Wang, M. Cao, P. Li, and Y.-j. Zhang, "Recognition of convolutional neural network based on cuda technology," *arXiv preprint arXiv:1506.00074*, 2015.
- [5] MPICH Development Team, *Mpich: High-performance portable mpi*, Accessed: Feb-2025, 2025. [Online]. Available: <https://www.mpich.org/>.
- [6] OpenMP Architecture Review Board, *Openmp application programming interface*, Accessed: Feb-2025, 2025. [Online]. Available: <https://www.openmp.org/specifications/>.
- [7] Wikipedia, 2025. [Online]. Available: <https://en.wikipedia.org/wiki/OpenMP>.